

Core Tensor Operations (Updated)

```
python
import torch
```

Copy

Tensor shape manipulation:

```
torch.unsqueeze() # Add a dimension of size 1 at specified position (e.g., [3] → [3, 1])
```

```
torch.squeeze() # Remove dimensions of size 1 (e.g., [3, 1] → [3])
```

Examples:

```
x = torch.tensor([1, 2, 3])
```

```
x_unsqueezed = x.unsqueeze(0) # Shape: [1, 3] (batch dimension)
```

```
x_squeezed = x_unsqueezed.squeeze(0) # Shape: [3]
```

Aliases (equivalent to unsqueeze/squeeze):

```
x.view(1, -1) # Reshape to add a dimension
```

```
x.reshape(1, -1) # Reshape flexibly
```

Other shape-related functions:

```
torch.expand_as() # Expand tensor to match another tensor's shape
```

```
torch.repeat() # Repeat tensor along specified dimensions
```

Why unsqueeze() and squeeze() Matter

- . **Broadcasting:** Adjust tensor dimensions for operations like $A + B$ where shapes differ.
Example: Add a vector $[3]$ to a matrix $[5, 3]$ by unsqueezing the vector to $[1, 3]$.
- . **Neural Network Layers:** Many layers (e.g., `nn.Conv2d`) require specific input shapes.
Example: Add a channel dimension to grayscale images with `unsqueeze(1)`.
- . **Batch Processing:** Add a batch dimension for single-sample inference.

Real-World Usage

```
python
```

Copy

Adding a batch dimension to a single image (for model input):

```
image = torch.randn(3, 28, 28) # Shape: [C, H, W]
```

```
image_with_batch = image.unsqueeze(0) # Shape: [1, 3, 28, 28]
```

Removing singleton dimensions after processing:

```
output = model(image_with_batch) # Shape: [1, 10]
```

```
output = output.squeeze(0) # Shape: [10]
```